

«Talento Tech»

# React JS

Clase 02



## Clase N° 2: JSX, props y componentes

### Índice

#### JSX, Props y Componentes a Fondo

##### Repaso javascript para react

- a) Destructuring (Desestructuración)
- b) Spread Operator (Operador de Propagación)
- c) Métodos de Array: .map()

##### Props en React:

Pasando datos entre componentes  
children : una prop "especial"

##### Patrón contenedor y presentacional: separando responsabilidades

Componentes Presentacionales (o "Dumb")  
Componentes Contenedores (o "Smart")

##### Formas de exportar mi componente

1. Exportación por Defecto (Default Export)
2. Exportación nombrada (Named export)
3. Exportación mixta

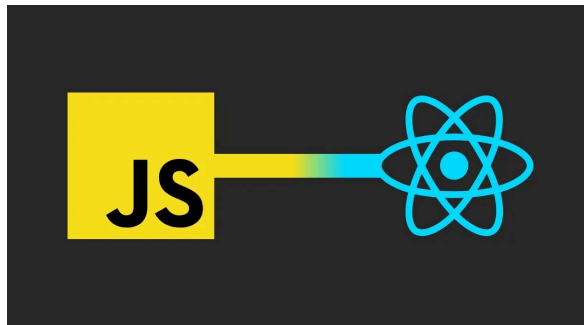
##### ¡Les damos la bienvenida a TalentoLab!

Ejercicio Práctico:  
Próximos Pasos:

### Objetivos de la Clase:

- Reforzar el uso y las ventajas de **JSX**.
- Repasar los **conceptos de JavaScript importantes** para el curso
- Entender a fondo cómo pasar datos con **props** y sus diferentes técnicas.
- **Organizar nuestros componentes** de forma prolija y escalable.
- Diferenciar entre componentes de **presentación** y componentes **contenedores**.

## Repaso javascript para react



### a) Destructuring (Desestructuración)

Imaginen que tienen un objeto con un montón de datos, pero solo les interesan algunos. La desestructuración es una forma rapidísima de sacar esos datos y guardarlos en variables. Cuando tenemos un objeto con varios datos de interes, como puede ser nuestro objeto usuario:

```
const usuario = {  
  nombre: "Ana",  
  edad: 28,  
  profesion: "Diseñadora UX"  
};
```

La forma que teníamos en Js de tomar cada uno de estos datos del objeto era de la siguiente manera:

```
// Forma tradicional  
const nombre = usuario.nombre;  
const edad = usuario.edad;
```

Con Destructuring vamos a generar en una línea las variables con los datos que queremos tomar de ese objeto de la siguiente manera:

```
// Con destructuring  
const { nombre, edad } = usuario;  
console.log(nombre); // "Ana"  
console.log(edad);   // 28
```

En React, lo vamos a usar todo el tiempo para recibir las props.



## b) Spread Operator (Operador de Propagación)

El spread operator (...) es como una varita mágica que nos permite "desparramar" los elementos de un array o las propiedades de un objeto dentro de otro. Es muy práctico para hacer copias o combinar datos sin modificar los originales.

Por ejemplo, cuando tenemos dos arrays con datos y los queremos unir, usamos el Spread Operator de la siguiente manera:

```
// Para arrays
const frutasRojas = ["frutilla", "cereza"];
const frutasAmarillas = ["banana", "mango"];
const todasLasFrutas = [...frutasRojas, ...frutasAmarillas];
// Resultado: ["frutilla", "cereza", "banana", "mango"]
```

Y cuando estamos trabajando con objetos, unimos todo de una forma similar, teniendo en cuenta que cuando una propiedad se repite, siempre se representa la última en se agregada.

```
const datosPersonales = {
  nombre: "Carlos",
  edad: 35
};

const datosProfesionales = {
  ...datosPersonales, // Copiamos todo lo de datosPersonales
  profesion: "Developer",
  edad: 36 // Si una propiedad se repite, la última pisa a la anterior
};
// Resultado: { nombre: "Carlos", edad: 36, profesion: "Developer" }
```

## c) Métodos de Array: .map()

Map nos permite recorrer un array y devolver un **nuevo array** con los elementos transformados. Es perfecto para renderizar listas de componente de forma dinámica. Es importante entender que **no se modifica el array original**.

En Javascript si tengo una lista de números y quiero obtener el doble de cada uno de esos números, "mapeo" el array y género un nuevo array donde a cada elemento lo multiplico por dos.

```
const numeros = [1, 2, 3, 4]; // Queremos duplicar cada número
const duplicados = numeros.map((num) => num * 2);
console.log(duplicados); // [2, 4, 6, 8]
```

En React nos queda de forma similar, donde el return devuelve cada list item (<li></li>) dentro de <ul> </ul>

```
const nombres = ['Matias', 'Sabrina', 'Luis'];  
// En React, lo usamos así para crear una lista de elementos:  
<ul>  
  {nombres.map( (nombre, index) => (  
    <li key={index}>{nombre}</li>  
  ))}  
</ul>
```

Vemos que además del nombre de cada elemento, obtengo el index o posición correspondiente a cada elemento.

**Ojo!** Siempre que usemos .map() para renderizar una lista en React, tenemos que tener en cuenta que cada elemento necesita una propiedad (llamada prop que veremos a continuación) especial llamada key. Esta key debe ser un valor único (como un ID del producto o el índice del array) para que React pueda identificar cada elemento de forma eficiente.

## Props: pasando datos entre componentes

En React, los componentes necesitan comunicarse entre sí.

La forma principal de hacerlo es a través de las props (abreviatura de properties).

👉 Podés pensarlas como los parámetros de una función, pero aplicados a los componentes.

Un componente padre le envía datos a un componente hijo a través de las props.

### Ejemplo básico

Imaginemos que tenemos un componente TarjetaUsuario que debe mostrar el nombre y la profesión de una persona. Veamos 3 formas que tenemos de comunicarnos entre componentes: 1. recibiendo la props completa, 2. con destructuring y 3. con Spread Operator.

Crearemos un componente llamado TarjetaUsuario.jsx . Estaremos trabajando en TarjetaUsuario.jsx y App.jsx para compartir los datos.

### Forma 1: Recibiendo el objeto props completo

```
// Componente TarjetaUsuario.jsx
export function TarjetaUsuario(props) {
  return (
    <div>
      <h2>{props.nombre}</h2>
      <p>{props.profesion}</p>
    </div>
  );
}
```

En App.jsx

```
function App() {
  return (
    <div>
      <Bienvenida /> {/* Clase anterior */}
      <p>Este es mi primer componente montado en App.jsx</p>
      <TarjetaUsuario nombre="Silvia" profesion="Product Owner" />
    </div>
  );
}
export default App;
```

✅ Lo bueno: es muy claro para empezar.

⚠️ Lo malo: escribir props. todo el tiempo puede volverse repetitivo.

¿Qué pasa si no le pasamos todos los datos? ¿Y si le pasamos uno de  $n$  props?

### Forma 2: Usando destructuring en los parámetros

La forma más usada y prolija. En lugar de recibir todo props, desestructuramos directamente lo que nos interesa.

```
// Componente TarjetaUsuario.jsx
export function TarjetaUsuario({ nombre, profesion }) {
  return (
    <div>
      <h2>{nombre}</h2>
      <p>{profesion}</p>
    </div>
  );
}
```

De esta manera nos evitamos de escribir las props cada vez que las necesitamos

### Forma 3: Usando el Spread Operator

Cuando ya tenemos un objeto con los datos que coinciden con las props, podemos pasarlos todos juntos. Con el mismo código en *TarjetaUsuario.jsx*, vamos a cambiar la forma en la que le pasamos las props. En *App.jsx* genero una variable llamada *usuario* donde le paso los datos que quiero mostrar. Luego el return devuelve el componente con el Spread Operator como props.

```
function App() {
  const usuario = {
    nombre: "Luis",
    profesion: "Diseñador UX/UI",
  };

  return (
    <div>
      <Bienvenida /> {/* Clase anterior */}
      <p>Este es mi primer componente montado en App.jsx</p>
      <TarjetaUsuario {...usuario} />
    </div>
  );
}

export default App;
```

👉 Esto es práctico cuando manejamos datos que vienen de una API o de una base de datos.

## children : una prop “especial”

Nosotros definimos todas las props que queremos que tenga nuestro componente, excepto una: **children**.

Children es una prop automática que contiene **todo lo que esté entre la etiqueta de apertura y cierre** de un componente. Para ver el funcionamiento de esta prop te propongo crear un nuevo componente llamado Contenedor.jsx y escribir el siguiente código:

```
// Componente Contenedor.jsx
function Contenedor({ children }) {
  const estilo = {
    border: "1px solid #ccc",
    padding: "16px",
    margin: "16px 0",
    "background-color": "red",
  };
  return <div style={estilo}>{children}</div>;
}
export default Contenedor
```

Luego lo importamos en App.jsx y lo usamos de la siguiente manera:

```
// /src/App.jsx
import Bienvenida from './Bienvenida';
import TarjetaUsuario from './TarjetaUsuario';
import Contenedor from './Contenedor'; /*Importo el componente*/
function App() {
  /* Código anterior*/
  return (
    /* Código anterior*/
    <Contenedor>
      <h1>Título dentro del contenedor</h1>
      <p>Un párrafo de ejemplo.</p>
    </Contenedor>
  </div>
  );
}
export default App;
```

👉 children es perfecto para crear componentes genéricos que envuelven a otros (layouts, cards, contenedores, etc.).



Habrás notado que la propiedad de CSS background-color no tiene el guion medio, por lo que fue reemplazado por el formato camelCase. Esto se debe a que Javascript no admite el guión medio. Todas las propiedades con esta característica se ven afectadas.

Entre ellas:

- background-color → backgroundColor
- font-size → fontSize
- text-align → textAlign
- border-radius → borderRadius



## Patrón contenedor y presentacional: separando responsabilidades

A medida que las aplicaciones crecen, es clave ser ordenados. Una de las mejores prácticas en React es separar los componentes en dos tipos:

### Componentes Presentacionales (o "Dumb")

- **¿Qué hacen?** Se preocupan únicamente por **cómo se ven las cosas**. No saben de dónde vienen los datos ni qué hacer con ellos.
- **Características:**
  - Reciben datos exclusivamente a través de **props**.
  - No manejan estado.
  - Son súper reutilizables.
- **Ejemplo:** Nuestra TarjetaUsuario es un componente presentacional perfecto. Solo muestra los datos que le llegan.

```
// Componente presentacional: solo se encarga de mostrar la info.
import React from "react";
function TarjetaUsuario({ nombre, profesion }) {
  return (
    <div>
      <h2>{nombre}</h2>
      <p>{profesion}</p>
    </div>
  );
}
export default TarjetaUsuario;
```

## Componentes Contenedores (o "Smart")

- **¿Qué hacen?** Son los cerebros de la operación. Se encargan de la **lógica**, de manejar el **estado** y de buscar los datos (por ejemplo, llamando a una API).
- **Características:**
  - Manejan la lógica de negocio.
  - Le pasan los datos a los componentes presentacionales.
  - No suelen tener mucho JSX propio, su trabajo es "orquestrar".
- **Ejemplo:** Un esqueleto de un contenedor que busca datos y luego los muestra.

Creemos un componente llamado **PerfilDeUsuarioContainer** (lo usaremos mas adelante). En él importamos el componente TarjetaUsuario (Dumb) y le pasamos los datos ya procesados, para que los presente.

```
import TarjetaUsuario from "../TarjetaUsuario";
// Componente contenedor: se encarga de la lógica.
export function PerfilDeUsuarioContainer() {
  // 1. 🧠 Acá declararíamos un "estado" para guardar los datos.

  // (Veremos `useState` en la próxima clase, por ahora imaginemos que funciona)
  const usuario = { nombre: "Matias", profesion: "Desarrollador" };

  // 2. 📄 Acá iría la lógica para obtener los datos (ej: una llamada a una API).

  // 3. ✅ Le pasamos los datos al componente presentacional para que los muestre.
  return (
    <TarjetaUsuario nombre={usuario.nombre} profesion={usuario.profesion} />
  );
}
```

Esta separación nos ayuda a mantener el código más limpio, fácil de testear y de mantener.

Habrás notado que este último componente era exportado en el mismo momento que se creaba la función... veamos las diferencias.

## Formas de exportar mi componente

Tenemos dos formas de exportar mi componente en React: Es importante reconocerlas porque difieren en como son importadas.

No vamos a crear código nuevo, solamente vamos a reconocerlo con el código que hicimos anteriormente y vamos a importarlo en nuestro archivo App.jsx.

1. Exportación por default
2. Exportación nombrada
3. Exportación Mixta



### 1. Exportación por Defecto (Default Export)

#### Características:

1. Solo puede haber uno por archivo
2. Se importa sin llaves
3. Puede tener cualquier nombre al importar

Ejemplo en TarjetaUsuario.jsx:

```
import React from "react";
// Componente presentacional: solo se encarga de mostrar la info.
function TarjetaUsuario({ nombre, profesion }) {
  return (
    <div>
      <h2>{nombre}</h2>
      <p>{profesion}</p>
    </div>
  );
}
export default TarjetaUsuario; /*Exportación por default*/
```

En app.jsx, donde estuvimos trabajando podemos importarlo de dos formas:

```
/* Otros componentes importados */
import Bienvenida from './Bienvenida';
import './App.css';
/* SIN LLAVES*/
import TarjetaUsuario from "./TarjetaUsuario"; /* nombre original*/
import MiTarjeta from "./TarjetaUsuario"; /* Cambio de nombre*/
```

## 2. Exportación nombrada (Named export)

### Características:

- Puede haber múltiples por archivo
- Se importa con llaves {}
- El nombre debe coincidir exactamente

Podemos verlo en **PerfilDeUsuarioContainer.jsx**

```
// Elimino los comentarios pero no hace falta que lo hagas
import TarjetaUsuario from "../TarjetaUsuario";
export function PerfilDeUsuarioContainer() {
  const usuario = { nombre: "Matias", profesion: "Desarrollador" };
  return (
    <TarjetaUsuario nombre={usuario.nombre} profesion={usuario.profesion} />
  );
}
```

Para importarlo, usamos las llaves en App.jsx de la siguiente manera:

```
// En /src/App.jsx
//Otras importaciones
import Contenedor from '../Contenedor';
import {PerfilDeUsuarioContainer} from "../PerfilDeUsuarioContainer";
import {PerfilDeUsuarioContainer as Usser} from "../PerfilDeUsuarioContainer";
// "apodo" Usser al componente PerfilDeUsusarioContainer
function App() {
  //Resto del código...
```

En el mismo archivo de PerfilUsuarioContainer.jsx creamos otra función de exportación nombrada. Para usarla, tenemos que llamarla desde App.jsx separada por coma dentro de las llaves. El código quedaría de la siguiente manera:

```
import {PerfilDeUsuarioContainer, OtroExport} from
"../PerfilDeUsuarioContainer";
```

### 3. Exportación mixta

Otra forma de exportar es combinando lo que necesitamos exportar.

#### Exportación Mixta

```
// PerfilDeUsuarioContainer.jsx
export default function PerfilDeUsuarioContainer() { // ← Default export
  // código principal
}
export function HelperFunction() { // ← Named export
  return "Función auxiliar";
}
export const config = { theme: "dark" }; // ← Otro named export
```

#### Importación combinada:

```
// En /src/App.jsx
import PerfilDeUsuarioContainer, { HelperFunction, config } from
"./PerfilDeUsuarioContainer";
// ↑ Default sin llaves ↑ Named con llaves
```

---

## Reflexión Final

Hoy profundizamos en los pilares que hacen a React tan poderoso: la capacidad de crear componentes reutilizables, comunicarlos con props y mantener el código ordenado con patrones como contenedor/presentacional. Dominar estos conceptos es clave para construir aplicaciones escalables y fáciles de mantener.

En la próxima clase, vamos a introducir uno de los hooks más importantes de React: useState, que nos permitirá darle memoria y estado a nuestros componentes. ¡Se pone cada vez mejor!



¡Les damos la bienvenida a TalentoLab!



En **TalentoLab**, convertimos ideas en herramientas digitales innovadoras y confiables, ofreciendo servicios de desarrollo con un enfoque en la calidad y eficiencia. Nuestro compromiso es desarrollar soluciones que optimicen procesos y potencien negocios, combinando creatividad, tecnología de punta y un compromiso absoluto con la satisfacción de nuestros clientes. Tu éxito es nuestra prioridad, y nuestras soluciones están diseñadas para marcar la diferencia.

#### Equipo TalentoLab:



**Silvia**  
Product Owner



**Luis**  
Diseñador UX UI



**Matias**  
Desarrollador



**Sabrina**  
Desarrolladora

## Situación Inicial en TalentoLab.

¡Únete al equipo TalentoLab! 🚀

La familia de **TalentoLab** está buscando un nuevo integrante para colaborar en el desarrollo de un emocionante producto innovador. Buscamos personas con un perfil **proactivo, autodidacta e innovador**, apasionadas por el aprendizaje continuo y con ganas de afrontar grandes desafíos.

Hemos preparado una serie de **ejercicios prácticos** diseñados para evaluar tus habilidades técnicas. Estos retos nos permitirán conocerte mejor y comprobar cómo aplicas tus conocimientos en situaciones reales.



### Ejercicio Práctico:

#### Lista de Asistentes

La idea es crear una pequeña aplicación que muestre una lista de asistentes a un evento. Para esto, vas a necesitar crear un componente reutilizable (Asistente) que muestre los datos de una persona y usarlo dentro de App.jsx para renderizar la lista completa a partir de un array de datos.

#### Pasos a seguir:

##### 1. Componente Asistente.jsx:

- Creá un nuevo archivo /src/Asistente.jsx.
- Este componente debe recibir tres props: nombre, tarea y emoji.
- Adentro, tiene que mostrar esos datos. Por ejemplo, en un div que contenga un h3 para el nombre y un p para la tarea con su emoji. ¡Usá la desestructuración de props que vimos!

##### 2. Componente App.jsx:

- Dentro de App.jsx, creá un array de objetos. Cada objeto va a representar un asistente y va a tener las propiedades nombre, tarea y emoji. Podés usar este de ejemplo

```
const asistentes = [
  { nombre: 'Juan Pérez', tarea: 'Frontend Developer', emoji: '💻' },
  { nombre: 'Ana Gómez', tarea: 'Diseñadora UX/UI', emoji: '🎨' },
  { nombre: 'Carlos Ruiz', tarea: 'Backend Developer', emoji: '👨‍💻' }];
```

Usá el método `.map()` sobre el array `asistentes` para generar un componente `<Asistente />` por cada persona en la lista.

No te olvides de pasarle a cada `<Asistente />` las props correspondientes y la key única.

## Materiales y Recursos Adicionales:

- ★ [Documentación oficial de React](#)
  - ★ [Guía sobre props en React](#)
- 

## Preguntas para Reflexionar:

- ¿Qué ventajas observas al usar props en componentes React?
  - ¿Cómo podrías reutilizar un componente en diferentes partes de una aplicación?
  - ¿Cuál es la diferencia entre un componente estático y uno dinámico que utiliza props?
- 

## Próximos Pasos:

- Aprender a estructurar un proyecto React desde cero.
- Crear componentes reutilizables que representen partes fundamentales de una interfaz.
- Aplicar estilos a nuestros componentes
- Comprender cómo ensamblar y renderizar estos componentes en el navegador.



**Buenos Aires**  
*aprende*  
Agencia de Políticas para el Futuro

**BA** Buenos  
Aires  
Ciudad